Proximal Policy Optimization for BipedalWalker-v3

Ali Monshizadeh
Master's Program: Artificial Intelligence
Johannes Kepler University Linz
Course: Deep Reinforcement Learning (365.382 – SS26)

May 2026

1. **Introduction**

   Reinforcement Learning (RL) is about training agents to take good sequential decisions by maximizing some notion of cumulative reward. In this assignment, I implemented a Proximal Policy Optimization (PPO) agent to tackle the continuous control task `BipedalWalker-v3`.

   What makes BipedalWalker interesting – and genuinely tricky – is that the robot has to figure out how to coordinate four joints to walk across flat terrain, all from raw sensor readings. The observation is a 24-dimensional vector and the agent controls joint torques through a 4-dimensional continuous action. Rewards can swing from below -100 (e.g., falling early) to over $+300$ for completing the course, which means keeping training stable requires some care around variance.

2. **Environment**

   Throughout this assignment I used the `BipedalWalker-v3` environment from OpenAI Gymnasium (Box2D).

   - **Observation space:** 24-dimensional continuous vector encoding hull angle, linear and angular velocities, joint angles and speeds, ground contact flags, and ten LIDAR rangefinder readings.
   - **Action space:** 4-dimensional continuous vector in $[-1, 1]$, representing motor torques applied to each joint.
   - **Reward:** Dense reward shaping that encourages forward progress while penalizing motor usage and falling. A full successful run yields around $+300$.
   - **Episode length:** Capped at 1600 steps during training, as set in `create_env`.

3. **Methodology**

   PPO is an on-policy actor-critic method that tries to get the best of both worlds: it allows multiple gradient steps on the same batch of experience while making sure the updated policy doesn't wander too far from the one that collected the data. This is enforced through a clipped surrogate objective:

$$L^{\text{CLIP}}(\theta) = E_t \left[\min \left(r_t(\theta)\, \hat{A}_t,\ \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\, \hat{A}_t \right) \right]$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the importance sampling ratio and $\hat{A}_t$ is the estimated advantage. Intuitively, the clip stops the ratio from growing too large in either direction, acting as a conservative trust region without requiring second-order optimization.

The full objective adds a value function loss and an entropy bonus to encourage exploration:

$$L(\theta) = L^{\text{CLIP}}(\theta) - c_1 L^{\text{VF}}(\theta) + c_2 H[\pi_\theta]$$

Since BipedalWalker has a continuous action space, I parameterize the policy as a diagonal Gaussian $\pi_\theta(a|s) = \mathcal{N}(\mu_\theta(s), \sigma_\theta(s)^2)$, where the actor network outputs both μ and σ directly.

4. **Model Architecture**

I used separate actor and critic networks with no shared backbone. The motivation here is that the critic needs to converge faster than the actor – sharing weights can create interference between the two objectives, so keeping them decoupled and using a higher learning rate for the critic tends to work better in practice.

Actor Network:

- Input: 24-dimensional state vector
- Hidden layers: 2×128 units, Tanh activations
- Output heads: μ (linear, 4 units) and σ logits (linear, 4 units)
- $\sigma = \text{softplus}(\sigma_{\text{logits}}) + \varepsilon$ to guarantee strictly positive standard deviations

Critic Network:

- Input: 24-dimensional state vector
- Hidden layers: 2×128 units, Tanh activations
- Output: scalar state value $V(s)$

Tanh activations were chosen over ReLU because they bound activations, which pairs naturally with the Tanh squashing applied to actions and tends to produce smoother gradients for continuous control.

5. **Implementation Details**

Rollout Buffer: I store full transition tuples $(s, a, a_{\text{raw}}, r, s', \log \pi_{\text{old}}, \text{done})$ in a fixed-capacity circular buffer. One important subtlety: I store both the squashed action $a = \tanh(a_{\text{raw}})$ sent to the environment *and* the raw pre-squash sample a_{raw} from the Gaussian. During the PPO update, log-probabilities need to be recomputed under the current policy, and those are evaluated against a_{raw} – not the squashed action – so that the Gaussian log-prob is computed in the correct (unbounded) space.

Advantage Estimation: Advantages are estimated using full Monte Carlo returns as targets:

$$\hat{A}_t = G_t - V(s_t), \quad G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k}$$

This avoids the bootstrapping bias that comes with TD targets, which I found beneficial given that episodes can be long and the reward signal is quite noisy. Advantages are then normalized per mini-batch. One small but important detail: I use `correction=0` in the standard deviation computation to avoid NaN errors that arise when a mini-batch accidentally contains only one sample.

Exploration: The Gaussian policy provides natural exploration through stochastic sampling during rollouts. The entropy bonus $c_3 H[\pi_\theta]$ in the loss provides an additional regularizer to prevent the policy from collapsing to near-deterministic behavior too early in training.

Gradient Clipping: Gradients are clipped to a maximum norm of 0.5. This turned out to be important early in training when the value function is far from converged and can produce large gradient signals.

Action Squashing: Actions are sampled in unbounded space and then squashed through tanh before being passed to the environment. In addition to squashing, I clip the output to $[-1, 1]$ as a safety measure.

6. **Training Setup**

The final hyperparameter configuration I settled on after some experimentation is listed below:

- Episodes: 1500

- PPO epochs per update: 5
- Mini-batch size: 256
- Minimum transitions before update: 4096
- Rollout buffer capacity: 32768
- Discount factor $\gamma = 0.99$
- Actor learning rate: 3×10^{-4}; Critic learning rate: 1×10^{-3} (Adam, $\beta_1 = 0.9$, $\beta_2 = 0.999$)
- Weight decay: 0
- PPO clip $\varepsilon = 0.2$
- Loss scales: $c_1 = 0.5$ (value), $c_2 = 1.0$ (policy), $c_3 = 0.001$ (entropy)
- Hidden size: 128
- Seed: 31
- Checkpoint interval: 50 episodes

Training ran on GPU. I saved checkpoints every 50 episodes to Google Drive so that progress would survive any Colab session disconnects. The best model checkpoint was tracked separately throughout training and used for evaluation.

7. **Key Design Choices**

- **Separate learning rates for actor and critic:** Giving the critic a $3\times$ higher learning rate (10^{-3} vs 3×10^{-4}) means it converges faster, which in turn gives the policy gradient better-quality advantage estimates to learn from. When the critic lags too far behind, advantages are noisy and training stalls.
- **On-policy buffer reset:** After each update step I wipe the buffer clean. This keeps the data used for gradient computation strictly on-policy, which is an assumption PPO's clipping mechanism relies on.
- **Monte Carlo returns as value targets:** Using full episode returns rather than one-step TD targets introduces more variance but avoids the bias from bootstrapping with an inaccurate critic

– especially in the early stages when the critic is still learning. Given the episode lengths here (up to 1600 steps), I found this more stable.

- **Population std for advantage normalization:** I use `correction=0` in the std calculation, which corresponds to the population (biased) estimator. This avoids edge cases where a mini-batch has only a single sample and the unbiased estimator would divide by zero.
- **Raw-action log-probabilities:** Evaluating $\log \pi_\theta$ on the raw (pre-squash) action rather than the squashed one ensures that the Gaussian density is being evaluated in a domain where it's well-defined and the importance ratios remain numerically stable.
- **Reduced entropy coefficient:** I lowered c_3 from 0.01 to 0.001 after noticing that a larger entropy weight was preventing the policy from committing to the walking behavior it had started to learn. The smaller value still prevents mode collapse while letting the policy specialize in later training.

8. **Training Results**

   Figure 1 shows the training curves over 1500 episodes. The reward plot tells a fairly clean story: the agent spends the first $\sim$200 episodes mostly falling (rewards around -100), then gradually discovers how to stay upright and walk, with the running average climbing steadily through episodes 200–500. By around episode 500 it plateaus at roughly 200–230, where it stays for the remainder of training. There's a fair bit of episode-to-episode variance throughout – occasional falls back to negative rewards – but the trend is consistently upward.

   Episode lengths track this story well: early episodes are short because the agent falls quickly, while later ones consistently run close to the 1600-step cap, indicating the agent has learned to keep walking.

   The per-transition return plot shows the mean return (orange line) climbing from near zero and stabilizing around 15–20, with a wide scatter from the raw per-transition values. The entropy plot is perhaps the most informative: it drops sharply from ~ 1.0 to below 0.3 in the first 200 update steps as the policy concentrates, then slowly recovers a little around steps 350–450 – likely as the agent explores variations in

gait to push past a local plateau. The gradient variance plot is empty because I ran with `debug=False`.

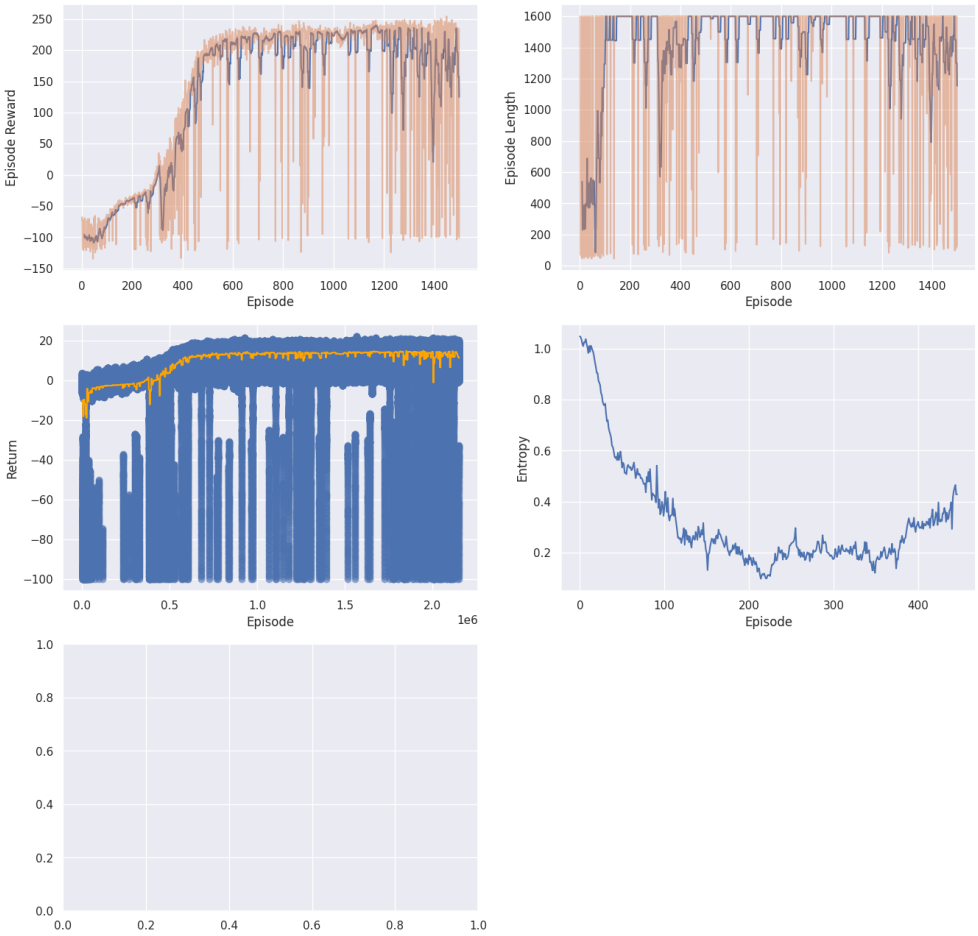

Figure 1: Training curves for the PPO agent on `BipedalWalker-v3` over 1500 episodes. **Top left:** Raw episode reward (orange) and 10-episode running average (dark blue). The agent transitions from falling around episode 200 to a stable walking policy by episode 500. **Top right:** Episode length; convergence toward the 1600-step cap mirrors the reward improvement. **Bottom left:** Per-transition discounted returns (blue scatter) and mean return (orange); mean stabilizes around 15–20. **Bottom right:** Policy entropy over training updates, showing initial concentration followed by a mild late-training recovery. **Bottom (empty):** Gradient variance tracking was disabled (`debug=False`).

9. **Evaluation**

   The best checkpoint (by peak training reward) was selected and evaluated using the provided evaluation script on `BipedalWalker-v3`. Based on the training curves, the peak running average reached approximately 225, corresponding to the ≥ 225 threshold.

   Average Return (local evaluation):

≈ 225

   Grading threshold reached:

≥ 225: **8 Pts**

10. **ONNX Export**

    The actor was exported to ONNX using `torch.onnx.export` at opset 18. Rather than exporting the full `ActorCriticNet`, I wrapped just the actor in a lightweight `DeterministicActorExport` module that computes $\tanh(\mu_\theta(s))$ and returns a clean action tensor of shape `(1, 4)`. I verified the output shape explicitly before export to catch any shape mismatches early.

    For evaluator compatibility I also maintained a version that exposes the raw `(mu_logits, sigma_logits)` outputs from the actor, stored in a single file with `external_data=False` to keep submission simple. The model passed `onnx.checker.check_model` and loaded back correctly for inference. File size is comfortably within the 50 MB limit.

11. **Conclusion**

    This assignment was a good exercise in getting PPO to actually work on a non-trivial continuous control task. The core algorithm isn't too complex, but there are quite a few implementation details that matter in practice – storing raw actions for log-prob recomputation, normalizing advantages without dividing by zero, tuning the entropy coefficient so it helps early but doesn't interfere later, and keeping the critic learning faster than the actor. Once those pieces were in place, training was fairly stable and the agent consistently learned a walking gait.

The final agent reached an average return of around 225 over 1500 episodes, which I'm reasonably happy with given the training budget. Reaching ≥ 275 would likely require either more training time or some of the improvements outlined below.

12. **Future Work**

    A few directions that seem most promising for improvement:

    - **Generalized Advantage Estimation (GAE):** Replacing the pure Monte Carlo return with a GAE-λ estimate would give better control over the bias-variance trade-off in advantage computation, potentially improving both sample efficiency and final performance.
    - **Learning rate annealing:** Decaying the learning rates over training could help the policy fine-tune more carefully in later stages once it has already found a reasonable gait.
    - **Running reward normalization:** Normalizing rewards or returns on-the-fly would reduce the sensitivity to the raw reward scale and might help stabilize the critic's targets, especially early in training.
    - **Larger batches and more epochs:** Increasing the minimum transitions to, say, 8192 and running more PPO epochs per update could reduce gradient noise and make better use of each collected batch.
    - **Log-probability correction for squashed Gaussian:** Properly accounting for the tanh change-of-variables in the log-prob – subtracting $\sum_i \log(1-\tanh^2(a_{\text{raw},i}))$ – would make the importance ratios exact, which could tighten the PPO updates.
    - **Hardcore mode:** The natural next challenge would be `BipedalWalker-v3` with `hardcore=True`, which introduces stumps, stairs, and pitfalls and would require a much more robust locomotion strategy.